Network Anomaly Detection Using Quantum Neural Networks on Noisy Quantum Computers

Kukliansky, Bollmann, Orescanin, Huffmire

Chiara Peppicelli

March 2026

UNIVERSITÀ DEGLI STUDI FIRENZE

Table of Contents

- Introduction
- Methodology
- Experiments and Results
- Conclusion

Motivation

- The rapid growth of cyber-attacks makes **network anomaly detection** a critical task.
- Traditional Intrusion Detection Systems (IDS):
  - Struggle with large-scale and complex network traffic.
- Machine Learning has improved Network Intrusion Detection Systems (NIDS). However:
  - Increasing data complexity.
  - Limitations of classical models.

$\Rightarrow$ **Can quantum neural networks be effective for network anomaly detection?**

Why QNNs for This Task?

- Network anomaly detection is a **binary classification problem** on complex traffic features.
- Quantum Neural Networks (QNNs) provide a quantum approach to supervised classification through **parameterized quantum circuits**.

Main limitation

— Current quantum devices operate in the **NISQ setting**.

— Limited qubits, noisy operations, and restricted circuit depth constrain performance.

$\Rightarrow$ The key issue is to balance **expressiveness** with **hardware feasibility**.

Reference Work

This project is inspired by the following paper:

Network Anomaly Detection Using Quantum Neural Networks on Noisy Quantum Computers - Kukliansky et al. (IEEE Transactions on Quantum Engineering, 2024).

The paper proposes:

- an efficient encoding of NetFlow features into qubits,
- several QNN architectures for anomaly detection,
- a certainty factor to assess prediction confidence,
- evaluation under realistic noisy quantum settings.

Table of Contents

2 Methodology

- Introduction
- Methodology
- Experiments and Results
- Conclusion

Dataset

- Experiments are conducted using the intrusion detection benchmark **NF-UNSW-NB15**.
- The dataset contains:
  - 2.39 million network flow records.
  - 12 original features.
  - binary labels: *benign* (-1) or *malicious* (1).
- Features related to IP addresses and ports were removed to:
  - reduce dimensionality.
  - avoid overfitting.

➥ The final dataset uses **8 numerical NetFlow features**.

Dataset Preprocessing

- The dataset was highly imbalanced, so benign samples were randomly downsampled to match malicious ones.
- Final **balanced dataset**:
  - **190,106** samples.
  - 95,053 benign.
  - 95,053 malicious.

Feature Name	Type	Range	Unique Values
IP Protocol	int	0 to 255	255
TCP Flags	int	0 to 31	15
Layer 7 Protocol	float	0 to 249	265
In Byte Count	int	52 to $2.68 \cdot 10^7$	12256
Out Byte Count	int	0 to $1.46 \cdot 10^7$	16047
In Packet Count	int	1 to 20038	892
Out Packet Count	int	0 to 11024	1207
Flow Duration [ms]	int	0 to $4.3 \cdot 10^6$	17377

Figure: Selected features from the NF-UNSW-NB15 dataset including the range of each selected feature and the number of its unique values.

Data Encoding

- Classical network features must be encoded into quantum states.
- The approach uses a **one-qubit-per-feature representation**.
- Each feature is encoded as a rotation around the X-axis:

$$|\psi\rangle = R_x(\theta)|0\rangle$$

- With:

$$|\psi\rangle = \cos\left(\frac{\theta}{2}\right)|0\rangle + \sin\left(\frac{\theta}{2}\right)|1\rangle$$

- Therefore **8 dataset features** $\rightarrow$ **8 qubits**.

Encoding Strategies

- **Categorical features:**
  - Used for features with a small number of unique values.
  - Each unique value is assigned an evenly spaced angle in $[0, \pi]$:

$$\theta_k = \frac{k}{N-1}\pi, \qquad k = 0, \ldots, N-1$$

- **Continuous features:**
  - Used for flow statistics such as byte counts, packet counts, and duration.
  - Values are assigned to percentile bins, if a sample falls into bin k out of B bins:

$$\theta = \frac{k}{B}\pi$$

Quantized Angle Mapping

2 Methodology

- A naive normalization into $[0, \pi]$ may produce extremely small rotations, problematic on noisy quantum hardware because they are highly sensitive to gate imperfections.
- To improve robustness, feature values are mapped to a **quantized angular range** with minimum granularity $0.25°$.

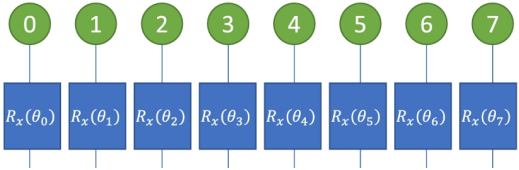


Figure: State preparation circuit used for the qubit encoding of the classical features.

QNN Architectures

Four QNN architectures are considered in this work:

- **Simple** variational circuit.
- **MERA** (Multi-scale Entanglement Renormalization Ansatz).
- **TTN** (Tree Tensor Network).
- **QCNN** (Quantum Convolutional Neural Network).

Common setting

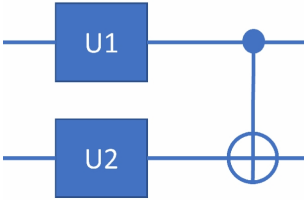
→ All architectures take encoded features as input.

→ Each model produces a single output qubit for binary classification.

Goal: Compare expressiveness, circuit depth, and robustness to noise.

Two-Qubit Building Block

- TTN, MERA, and QCNN are constructed using a common **two-qubit block**.

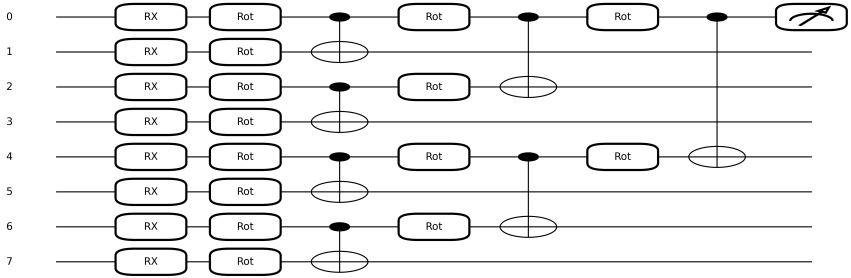


- **6 trainable parameters** (3 per qubit).
- Single CNOT reduces circuit depth and improves robustness to noise.
- **Trade-off** between reduced expressiveness and increased robustness under noisy conditions.

Tree Tensor Network (TTN)

2 Methodology

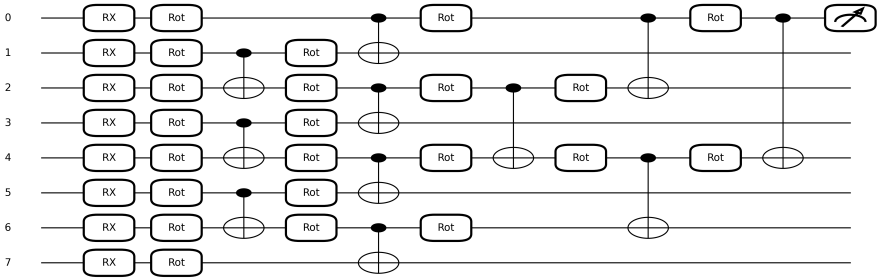
Hierarchical tree-structured architecture with qubits processed in pairs.



The number of active qubits is progressively reduced until a single output qubit remains.

MERA Architecture

Hierarchical architecture extending TTN with additional layers to capture multi-scale correlations.

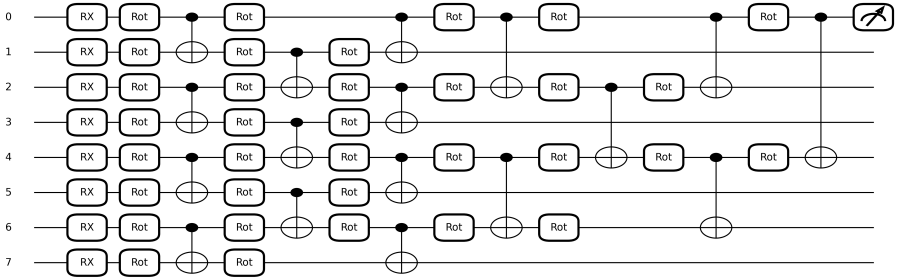


Alternates **disentanglers** and **isometries**, enabling both local and long-range interactions.

Quantum Convolutional Neural Network (QCNN)

2 Methodology

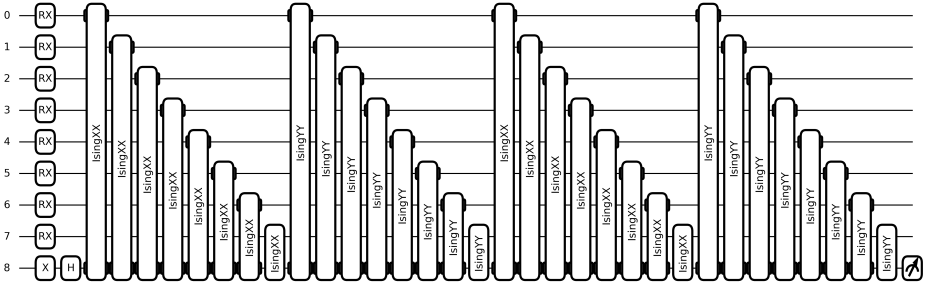
Quantum analogue of CNNs, combining convolution and pooling operations.



Applies **parameter sharing** and progressively reduces qubits through pooling layers.

Simple Architecture

Star-shaped architecture with a dedicated result qubit interacting with all feature qubits.



Built using **Pauli interaction gates** (R_{XX}, R_{YY}, R_{ZZ}) applied in multiple layers.

Certainty Factor

2 Methodology

- The paper introduces the **certainty factor** as a **signed confidence measure**.
- For an output state

$$|\psi\rangle = \alpha_0|0\rangle + \alpha_1|1\rangle$$

  and expected class $|0\rangle$, it is defined as

$$s(x) = |\alpha_0|^2 - |\alpha_1|^2 = p_0 - p_1, \qquad C \in [-1, 1]$$

- In the implementation, the certainty factor is computed as a **label-aligned signed output**:

$$C(x) = y_{\pm 1}\, s(x), \qquad y_{\pm 1} \in \{-1, +1\}$$

- **Interpretation**:
  - $C(x) \approx 1$: correct and highly confident prediction.
  - $C(x) \approx 0$: uncertain prediction, close to the decision boundary.
  - $C(x) \approx -1$: highly confident but wrong prediction.

Table of Contents

3 Experiments and Results

- Introduction
- Methodology
- Experiments and Results
- Conclusion

Experimental Setup

- All experiments were carried out using **PennyLane**.
- Multiple runs were performed by varying:
  - optimizer, learning rate, batch size, and, for the Simple model, circuit depth and layer configuration.
- The balanced dataset was split using an **85% / 15%** train-test ratio. In addition, **15% of the training set** was used as a validation set.
- Evaluation metrics:
  - Accuracy.
  - Precision.
  - Recall.
  - F1-score.
  - ROC-AUC.

Architecture Comparison

Main result

In the noiseless setting, the **Simple** architecture achieved the best overall performance. **QCNN** ranked second, while **TTN** and **MERA** obtained very similar results.

- The main reference metric is the **F1-score**, since anomaly detection requires balancing false positives and false negatives.
- The **Simple** model achieved the highest **test accuracy** and **F1-score**.

Architecture	Test Acc.	F1-score
TTN	0.7430	0.7373
MERA	0.7445	0.7392
QCNN	0.7641	0.7640
Simple	**0.7805**	**0.7972**

Table: Best noiseless accuracy and F1-score obtained for each architecture.

Architecture Comparison

Architecture	Precision	Recall	ROC-AUC
TTN	0.7540	0.7214	0.7055
MERA	0.7547	0.7243	0.7114
QCNN	**0.7642**	0.7637	0.7654
Simple	0.7411	**0.8625**	**0.8057**

Table: Best noiseless precision, recall, and ROC-AUC obtained for each architecture.

- **QCNN** achieved the highest **precision**.
- **Simple** achieved the highest **recall** and **ROC-AUC**.
- For anomaly detection, this makes the **Simple** model the strongest overall choice.

$\Rightarrow$ The best Simple model is not the most precise, but it is the most effective overall.

Focus on the Simple Architecture

- Among all tested models, the **Simple architecture** achieved the best noiseless performance.
- For this reason, a more detailed analysis was carried out on this model.
- The study focused on:
  - number of layers,
  - interaction pattern,
  - optimizer,
  - learning rate,
  - batch size.
- **Goal:** identify the most effective configuration before moving to the noisy setting.

Simple Architecture: Structural Exploration

4 Noiseless Experiments

Effect of circuit depth

Layers	Best Acc.	Best F1	Mean F1
1	0.7642	0.7641	0.7442
2	0.7628	0.7748	0.7445
4	0.7489	0.7691	0.7525
6	**0.7805**	**0.7972**	**0.7596**

Effect of interaction pattern

Type	Best Acc.	Best F1	Mean F1
XXYY	**0.7805**	**0.7972**	**0.7606**
ZZXX	0.7642	0.7641	0.7566
ZZYY	0.7596	0.7691	0.7320
ZZXXYY	0.7642	0.7641	0.7343

$\Rightarrow$ The strongest structural configuration is the **6-layer XXYY** Simple model.

Simple Architecture: Optimization Setup

Optimizer and learning rate

Optimizer	LR	Best Acc.	Best F1
Adam	0.010	0.7606	0.7723
Adam	0.020	0.7694	0.7785
Adam	**0.050**	**0.7805**	**0.7972**
Adam	0.100	0.7721	0.7822
SGD	0.010	0.7564	0.7675
SGD	0.020	0.7575	0.7688
SGD	0.100	0.7583	0.7702

Batch size

Opt.	Batch	Acc.	F1
Adam	16	0.7721	0.7822
Adam	**32**	**0.7805**	**0.7972**
SGD	16	0.7583	0.7702
SGD	32	0.7575	0.7688

- **Adam** consistently outperformed **SGD**.
- The best result was obtained with **learning rate 0.05** and **batch size 32**.

Best Noiseless Configuration

Model structure

- Architecture: **Simple**
- Layer pattern: **XXYY**
- Number of layers: **6**

Training setup

- Optimizer: **Adam**
- Learning rate: **0.05**
- Batch size: **32**

Best test performance

- Accuracy: **0.7805**
- F1-score: **0.7972**
- Recall: **0.8625**
- ROC-AUC: **0.8057**

Certainty Factor in the Noiseless Setting

Definition

For the certainty analysis in this work, the certainty factor is computed as a **label-aligned signed margin**:

$$\mathcal{C}(x) = y_{\pm 1}\, s(x), \qquad \mathcal{C}(x) \in [-1, 1]$$

$|\mathcal{C}(x)| \approx 1$: confident prediction; $|\mathcal{C}(x)| \approx 0$: sample close to the decision boundary.

- Across all tested architectures, the certainty distributions are mostly **well separated from zero**.
- Correctly classified samples are associated with **positive certainty values**.
- Incorrect predictions are concentrated on the **negative side**.

$\Rightarrow$ In the noiseless setting, the certainty factor is informative about both **prediction correctness** and **distance from the decision boundary**.

Noiseless Certainty: Quantitative Comparison

Model	mean $\lvert\mathbf{C}\rvert$	$\lvert\mathbf{C}\rvert < 0.1$	$\lvert\mathbf{C}\rvert < 0.2$	$\lvert\mathbf{C}\rvert < 0.5$
TTN	0.7420	0.42%	0.87%	31.58%
MERA	0.7476	0.43%	0.83%	31.54%
QCNN	0.778	0.00%	0.01%	29.38%
Simple	0.6466	5.68%	10.10%	24.45%

- **QCNN** shows the sharpest certainty distribution overall.
- **TTN** and **MERA** have very similar certainty profiles.
- The best **Simple** model is less polarized, with more samples close to the boundary, but it still maintains a meaningful certainty structure.

Noiseless Certainty: Violin Plots

4 Noiseless Experiments

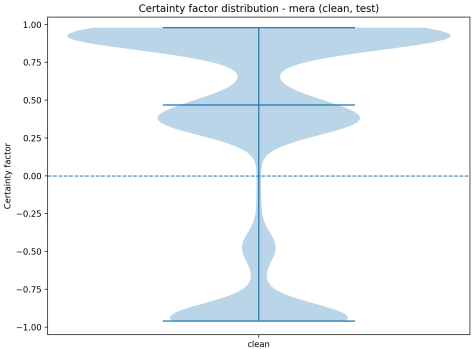


Figure: Certainty-factor distribution for the MERA architecture.

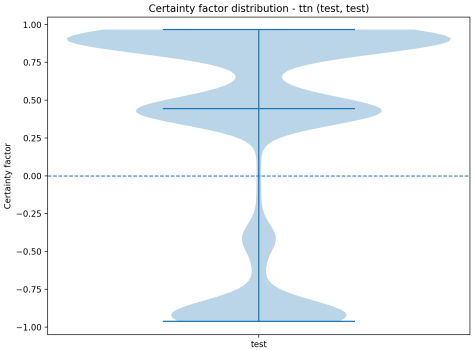


Figure: Certainty-factor distribution for the TTN architecture.

Noiseless Certainty: Violin Plots

4 Noiseless Experiments

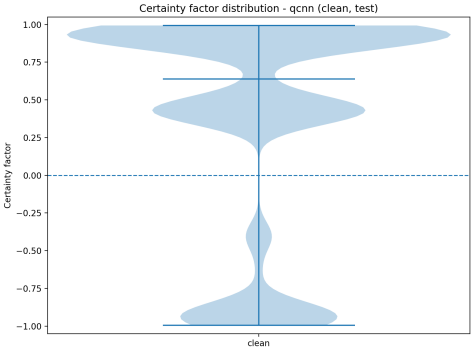


Figure: Certainty-factor distribution for the QCNN architecture.

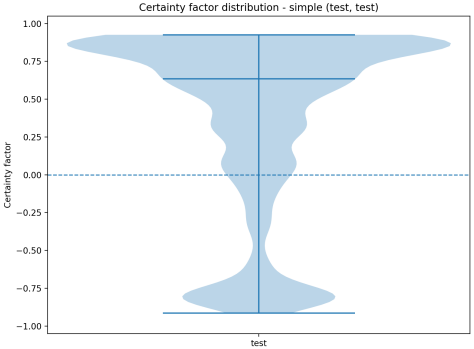


Figure: Certainty-factor distribution for the Simple architecture (6 layers, XXYY).

Decisiveness Does Not Equal Accuracy

Model	$\mathbb{E}[\mathbf{C} \mid \mathbf{correct}]$	$\mathbb{E}[\mathbf{C} \mid \mathbf{wrong}]$	F1-score
TTN	0.7164	-0.8156	0.7373
MERA	0.7161	-0.8383	0.7392
QCNN	0.7534	-0.8593	0.7640
Simple	0.6511	-0.6307	**0.7972**

- QCNN, TTN, and MERA produce more **extreme outputs**, even when they are wrong.
- The best Simple model is **less extreme on average**, but achieves the best classification performance.

Key takeaway

The certainty factor captures **decision sharpness**, but sharper outputs do not necessarily translate into better classification performance.

Noisy Simulation Setup

- The noisy study uses the **Simple model** with the **best noiseless configuration** (**XXYY**), but with **2 layers** instead of 6, to make the full noise sweep computationally feasible.
- Noise is modeled through **depolarizing channels** applied to both:
  - single-qubit gates.
  - two-qubit gates.
- Considered noise regimes:

Clean ($p = 0$)	**Very low** ($p = 0.001$)	**Low** ($p = 0.005$)
Medium ($p = 0.01$)	**High** ($p = 0.05$)	**Very high** ($p = 0.1$)

Goal: analyze how increasing noise affects both **classification performance** and **prediction certainty**.

Classification Performance Under Increasing Noise

5 Noisy Experiments

Noise	Acc.	F1	Prec.	Rec.	ROC-AUC
Clean	0.7628	0.7748	0.7391	0.8079	0.7968
Very low	0.7640	0.7790	0.7389	0.8238	0.7935
Low	0.7680	0.7824	0.7433	0.8257	0.7892
Medium	0.7560	0.7715	0.7318	0.8158	0.7833
High	0.5600	0.6000	0.5546	0.6535	0.5780
Very high	0.4930	0.5301	0.4983	0.5663	0.4936

- Up to **medium noise**, performance remains relatively stable.
- A sharp degradation appears only in the **high** and **very-high** regimes.

Certainty Factor Degradation Under Noise

5 Noisy Experiments

Noise	$\mathbb{E}[\lvert\mathbf{C}(\mathbf{x})\rvert]$	$\lvert\mathbf{C}\rvert < \mathbf{0.2}$	$\lvert\mathbf{C}\rvert < \mathbf{0.5}\}$
Clean	0.6807	6.5%	23.4%
Very low	0.6425	6.6%	25.0%
Low	0.5074	9.3%	42.9%
Medium	0.3826	15.9%	73.1%
High	0.1156	78.8%	100.0%
Very high	0.1127	80.1%	99.8%

- The average certainty magnitude decreases **steadily** as noise increases.
- At the same time, the fraction of low-certainty predictions grows rapidly.

Noisy Certainty: Violin Plots

5 Noisy Experiments

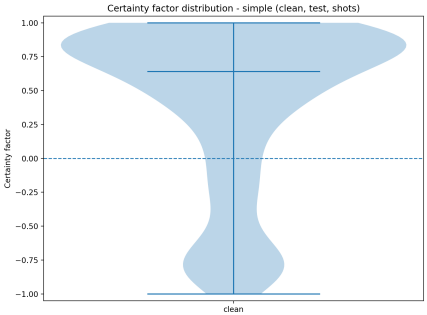


Figure: Violin plot of the certainty-factor clean setting.

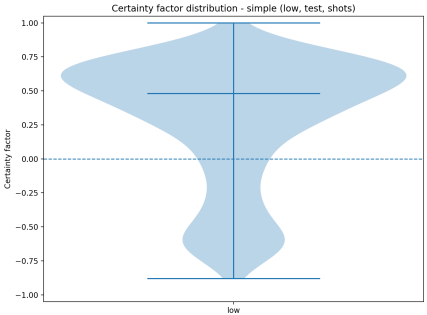


Figure: Violin plot of the certainty-factor under low noise.

Noisy Certainty: Violin Plots

5 Noisy Experiments

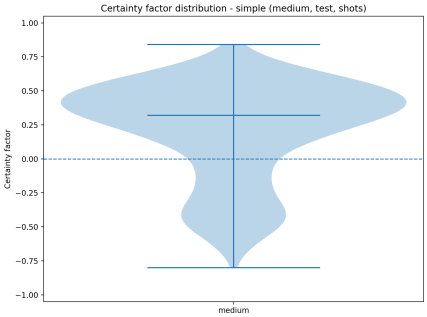


Figure: Violin plot of the certainty-factor under medium noise.

Figure: Violin plot of the certainty-factor under high noise.

Noisy Certainty: Violin Plots

5 Noisy Experiments

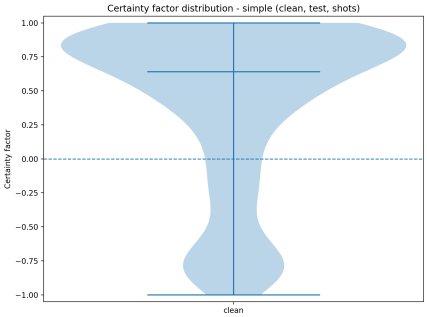


Figure: Violin plot of the certainty-factor under no noise.

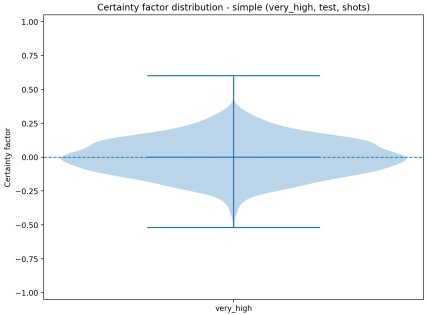


Figure: Violin plot of the certainty-factor under very high noise.

How Noise Degrades the Classifier

- The certainty factor reveals the degradation process more clearly than hard-label metrics.
- At **moderate noise**, the certainty distribution is more compressed toward zero:
  - prediction margins become smaller,
  - while F1-score can still remain relatively stable.
- At **high noise**, the distribution collapses around zero:
  - most samples become low-confidence,
  - class separation is largely lost,
  - and performance approaches random guessing.

$\Rightarrow$ Noise first reduces **decision margins and certainty**, and only at stronger levels produces a clear collapse in **classification performance**.

Table of Contents

6 Conclusion

- Introduction
- Methodology
- Experiments and Results
- Conclusion

Main Findings

- Among the tested QNNs, the **Simple architecture** achieved the best overall performance in the noiseless setting.
- This suggests that, for this task, a simpler design can be highly effective even compared with more structured architectures such as TTN, MERA, and QCNN.
- The **certainty factor** provided a useful complementary perspective:
  - it separated **correct** and **incorrect** predictions by sign,
  - it measured the distance from the decision boundary through $|\mathcal{C}(x)|$,
  - and it revealed robustness loss earlier than F1-score or accuracy alone.

Conclusion

- This work explored the use of **Quantum Neural Networks** for **network anomaly detection**.
- The results indicate that QNN-based classifiers can be effective in **ideal simulation**, showing that this approach is promising for the task.
- However, the noisy analysis shows that performance and confidence degrade progressively as noise increases, highlighting the importance of **robustness** for practical quantum implementations.

UNIVERSITÀ DEGLI STUDI FIRENZE

Network Anomaly Detection Using Quantum Neural Networks on Noisy Quantum Computers

Chiara Peppicelli

Thank you for listening!

Appendix: References

6 Conclusion

A. Kukliansky, M. Orescanin, C. Bollmann, and T. Huffmire, “Network Anomaly Detection Using Quantum Neural Networks on Noisy Quantum Computers,” *IEEE Transactions on Quantum Engineering*, vol. 5, pp. 1–11, 2024. doi: 10.1109/TQE.2024.3359574.